



Reporte HPC

Pedro Ismael Guerrero Enciso
*Computo de Alto Rendimiento,
Posgrado en Ciencias (Física), 2026-2
Universidad Nacional Autónoma de México*

Resumen

El presente reporte hace un breve recuento de las estrategias de paralelización usadas en el código *calor2D.f90*, para el caso de dos CPU de diferentes marcas y con diferentes enfoques, haciéndose especial énfasis en como la diferencia de arquitectura afecta el rendimiento. Se encontró que el CPU de escritorio de AMD es superior al CPU de laptop de Intel, incluso cuando se trata de consumo energético.

(Fecha de entrega: 5/06/2026)

1. INTRODUCCIÓN

En el ámbito del computo científico de alto rendimiento se pueden identificar 3 puntos clave que determinaran completamente la lógica de programación a seguir y las herramientas a emplear, aún antes de siquiera comenzar a escribir.

El primero y más básico a considerar es la arquitectura en la que se espera que el código se ejecute. Históricamente se ha trabajado con set de instrucciones x86 creado por Intel en 1978, y posteriormente extendida para trabajar con 64 bits por AMD [1], como el estándar en CPU. Sin embargo, el auge de los chips móviles, su entrada al mercado de laptops con el Apple Silicon M1 en 2020 [2], y el más reciente anuncio de los chips NVIDIA VERA para servidores [3], pone a la arquitectura ARM como una opción viable para el computo de alto rendimiento. En particular en dinámica de fluidos computacional, se ha visto con el port de OpenFOAM para apple silicon permite rendimientos muy buenos incluso en mallas grandes gracias a su memoria unificada[4].

Por otro lado tenemos a las GPU, que al no contar con un conjunto de instrucciones estandarizado (ISA, por sus siglas en inglés), limita la portabilidad de código entre GPUs de diferentes desarrolladores. Sin embargo, existen proyectos open source que tratan de mitigar lo anterior, tal es el caso de SCALE [5] que busca ser un compilador universal que permita la ejecución de código nativo de CUDA en cualquier GPU.

Lo anterior condiciona directamente la elección del sistema operativo. Si bien esto no genera inconvenientes cuando se realiza código para CPU, en el caso de las GPU puede ser realmente problemático. Como ejemplo tenemos el caso de AMD RCOM, que cuenta con soporte oficial para Ubuntu, RHEL, SLES, Debian, Rocky Linux, Oracle Linux y Windows [6], pero sólo para versiones específicas y sin brindar una versión general, como si lo llega a hacer Nvidia con su HPC SDK [7]. Por ese motivo el soporte en otras distribuciones depen-

de de la comunidad, lo que puede generar problemas como los existentes en Arch Linux, donde los compiladores de Fortran simplemente no funcionan. limitando así el uso de dicho lenguaje dentro de ese sistema operativo.

Una vez elegido el lenguaje de programación, es igualmente importante prestar atención a como administra la memoria, es decir, si se trata de un lenguaje *row-major* o *column-major*, ya que ignorar este hecho puede llevar a cuellos de botella en el código, que destruyen el rendimiento aún en un código secuencial.

Posteriormente, es necesario entender como gestionan las API la memoria dentro de las regiones paralelas, con el objetivo de determinar que variables deben ser privadas (cada hilo tendrá su propia copia) y cuales publicas (todos comparten la variable). Si bien, las APIs pueden hacer la distinción por si mismas, generalmente es mejor indicar explícitamente que tipo de variables se utilizarán.

Con lo anterior ya seleccionado, es importante también conocer la configuración del sistema. Los procesadores cuentan con diferentes frecuencias dependiendo de la carga de trabajo que ejecuten, por lo cual, a menos que se cuente con un sistema de enfriamiento adecuado, el sistema puede presentar *thermal throttling*, lo que limita la escalabilidad del código.

Adicionalmente tenemos el ejemplo de los procesadores de Intel, que a partir de su 12ava generación (*Alder Lake*) implementaron un esquema híbrido en sus chips, con núcleos de alto rendimiento (P-cores) y núcleos de bajo consumo (E-cores) [8], lo que mejora considerablemente la duración de la batería y reduce el calor generado por los chips, sin embargo, en aplicaciones de alto rendimiento el escalamiento, tal como se mostrará más adelante, no es tan adecuado, amenos que se le añadan las optimizaciones pertinentes al código.

Es así que podemos identificar un mínimo de 3 fases en la lógica de programación a seguir:

1. Fase de conceptualización: dónde se identifican y de-

finen la arquitectura a emplear, el sistema operativo y el lenguaje de programación. De igual manera se complementa con diagramas de flujo y pseudo-códigos que sirven de guía.

2. Implementación inicial: Se realiza una primera iteración del código basada en el pseudo-código previamente escrito.
3. Debuggin y Optimización: Lo importante en esta fase es comparar casos de estudio conocido para verificar el funcionamiento adecuado del algoritmo, así como la obtención de métricas que permitan mejorar el tiempo de ejecución del código y optimizar el consumo de energía.

Con todo lo anterior dicho, el enfoque de este trabajo será comparar el mismo código, calor2D, escrito en *Fortran 90* con *OpenMP* como la API de paralelización para 2 CPU x86. El primero con una arquitectura homogénea y el segundo con una híbrida [9, 10]. Las especificaciones de ambos modelos de CPU, así como de los equipos donde se encuentran montados se resumen en la tabla siguiente. Cabe resaltarse que se asumió que el uso de Hyper-Threading incrementa el consumo en un 10 % con respecto a los núcleos físicos en ambos casos.

Configuraciones a comparar		
Modelo	AMD Ryzen 7 5700x3D	Intel i5 1335u
No. y Tipo de Núcleos e Hilos	8 Núcleos, 16 Hilos	10 Núcleos (2 P y 8 E) y 12 Hilos
Frecuencia Base	3 GHz	E-Core: 0.9 GHz, P-Cores: 1.3 GHz
Frecuencia Turbo	4.1 GHz	E-Core: 3.4, GHz P-Cores: 4.6 GHz
Cantidad de Caché	96 MB	12 MB
Consumo Máximo (W)	142	55
Consumo por Núcleo usando Hyper-Threading (W)	17.75	P-Core: 17.5, E-Cores: 2.5
Consumo por Núcleo sin Hyper-Threading (W)	15.975	P-Core: 15.75, E-Cores: 2.5
Sistema de Enfriamiento	Noctua NH-D14	Dual-vented exhaust de ASUS
RAM Empleada	32 GB DDR4 3200 MT/s	8 GB DDR4 3200 MT/s
Sistema Operativo	Cahyos LTS (Arch Linux)	Ubuntu 24.04.4 LTS

Tabla I. Especificaciones de CPU

1. Código a analizar

Para realizar el análisis se ejecutó el código *calor2D.f90*, cuyo objetivo es resolver la ecuación de Laplace en 2 dimensiones mediante un método iterativo, partiendo de una discretización en diferencias finitas. La paralelización de realizó haciendo uso de la API de OpenMP, que permite su implementación de forma casi directa sobre código secuencial, sin necesidad de realizar modificaciones importantes dentro del código.

De manera general el algoritmo cuenta con un bucle principal de iteraciones, encargado de evolucionar los valores de temperatura al interior de la malla. Al interior existen dos bucles de iteraciones que se encargan de realizar los barridos de manera independiente en cada dirección (x,y). Es sobre estos bucles donde se realizó la paralelización, permitiendo que el paralelismo sea de grano grueso, ideal para CPUs.

Si bien, dada su estructura lo anterior puede introducir condiciones de carrera, especialmente en la parte asociada a la inversión de la matriz, la naturaleza iterativa del código nos permite ignorarlos sin mayor problema, ya que se espera que

el código llegue a un estado estacionario determinado. Sin embargo, para su correcta implementación es necesario identificar que variables serán privadas y cuales compartidas entre los hilos del CPU.

Al interior de cada bucle de barrido encontramos un bucle principal, encargado de ensamblar la matriz a invertir. Esto se hace mediante el uso de 4 arreglos unidimensionales. Acto seguido se aplican las condiciones de frontera sobre estos arreglos, para terminar la sección invirtiendo la matriz y actualizando los datos de temperatura.

Finalmente se cuenta con un bucle encargado de calcular el residuo, que nos permite identificar cuando se ha llegado al resultado deseado y marca el punto a partir del cual podemos detener el algoritmo iterativo a sabiendas de que el resultado es correcto.

2. MÉTODO Y RESULTADOS

Para realizar una comparativa entre diferentes configuraciones de hilos se modificó la variable de entorno *OMP_NUM_THREADS*, variando la desde 2 hasta el número máximo de hilos del CPU. Mientras que los tiempos obtenidos para el tiempo secuencial se obtuvieron eliminando la bandera *-fopenmp* al momento de compilar el código. Para un número de hilos determinado se realizaron 10 iteraciones, cuyos tiempos fueron promediados. Esos promedios fueron los datos empleados para la obtención de las gráficas posteriores.

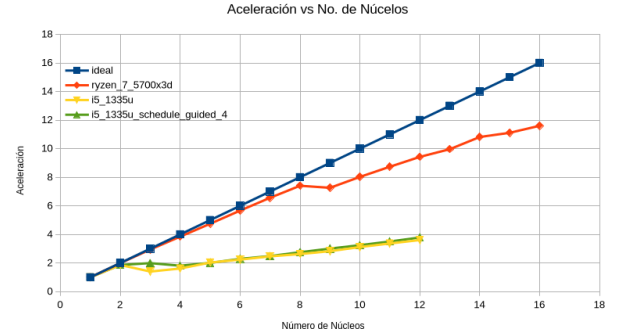


Fig. 1. Aceleración.

La aceleración (fig: 1) es una de las primeras métricas a obtener cuando se trata de realizar un análisis sobre el rendimiento de un determinado código. La operación es bastante simple, y consiste en dividir el tiempo paralelo para un determinado número de hilos entre el tiempo en secuencial. Idealmente el código puede alcanzar una aceleración igual al número de hilos empleados. Sin embargo ese casi nunca es el caso, debido a que abrir regiones paralelas y generar copias privadas requiere de tiempo, lo que puede llegar a entorpecer el código. Por lo que un balance entre la carga de trabajo y el uso de memoria es necesario.

El tiempo de computo (fig:2) nos brinda en cierto sentido la misma información que la aceleración, con la diferencia clave que nos da una referencia visual de cuanto tiempo realmente

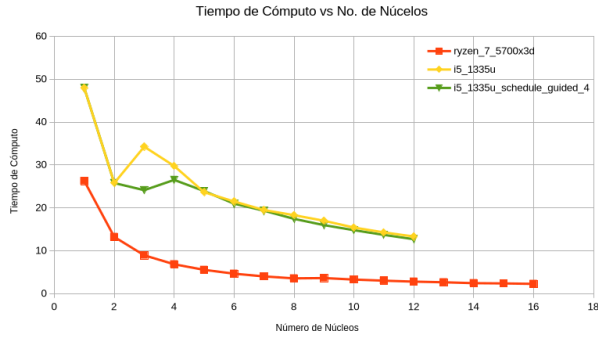


Fig. 2. Tiempo de Cómputo.

se esta ganando al incrementar el número de núcleos e hilos ejecutando el código.

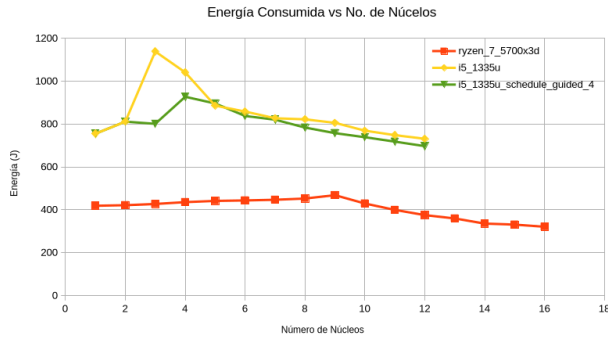


Fig. 3. Consumo de Energía

Por otra parte, el consumo de energía (fig:3) es otra métrica interesante, ya que nos permite cuantificar si la reducción en el tiempo de computo realmente es beneficiosa. En este caso los datos se obtuvieron multiplicando el tiempo que dura la simulación por el consumo de cada núcleo en Watts (tab:I). Para el caso el procesador de Intel se asumió que el paso de núcleos físicos a lógicos para los P-cores ocurre cuando pasamos de 2 a 3 núcleos.

A su vez, obtener las diferencia porcentuales de aceleración (fig:4) y consumo (fig:5) nos facilita identificar si el aumento o disminución en el consumo están ligados a la aceleración dentro del código y que tan grande es respecto al paso de tiempo anterior, de igual manera ayudan a identificar el balance optimo entre tiempo y consumo. Dichas métricas se obtiene mediante la ecuación:

$$\Delta x = \frac{x_n - x_{n-1}}{x_{n-1}} * 100, \quad (1)$$

con x la variable a analizar.

3. DISCUSIÓN Y CONCLUSIONES

La figura 1 nos permite identificar claramente el cambio entre núcleos físicos y núcleos lógicos en el caso del ambos

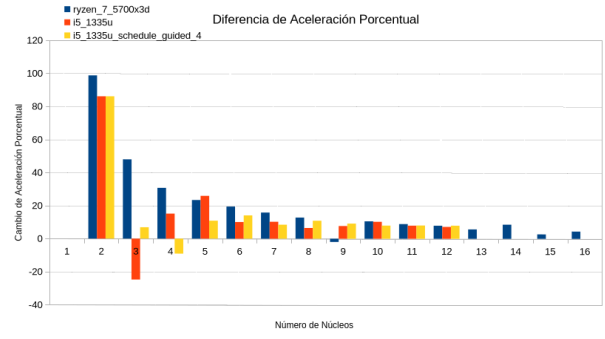


Fig. 4. Diferencia en Aceleración

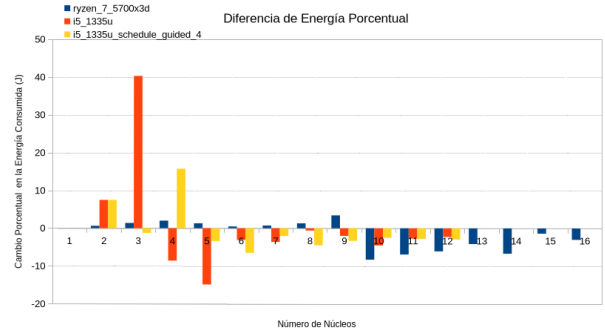


Fig. 5. Diferencia en Energía

CPU. En ambos casos se presenta una aceleración bastante cercana a la ideal cuando ocupan los núcleos físicos (1 a 8 para el Ryzen y 2 en el caso del i5). Sin embargo, es claro que existe una gran caída en el rendimiento cuando se comienzan a implementar los núcleos lógicos, como se corrobora en el núcleo 9 del procesador de AMD, donde la aceleración de hecho llega a ser negativa, siendo un 1.9 % inferior (fig:4) y generando un consumo 3.4 % mayor (fig:5) que en el caso de 8 núcleos.

Sin embargo, cuando se implementa más de 1 solo núcleo lógico, la aceleración vuelve a aumentar y el consumo baja. De hecho, tal es el caso que en general el consumo de energía total mejora al hacer uso de los núcleos lógicos, llegando a consumir incluso menos que en el caso secuencial. Por lo que de ser posible es recomendable hacer uso de los núcleos lógicos, ya que incrementan la aceleración apenas aumentando el consumo por núcleo como se reporta en la tabla I.

Si bien la aceleración para 2 núcleos en ambos CPUs es casi idéntica, con ambos dividiendo el tiempo de computo en casi a la mitad, la realidad que la gráfica 2 nos permite vislumbrar, es que la diferencia en el tiempo de computo entre un procesador y el otro es de 20 segundos para el caso secuencial y 15 para el caso paralelo. Lo anterior se traduce en básicamente el doble del consumo de energía durante la simulación. Eso a pesar de que el consumo por núcleo del procesador de Intel es ligeramente menor.

En general, no es una sorpresa que el Ryzen 5700x3D sea un mejor procesador en todos los aspectos que el i5 1335u, pero esto no quiere decir que el procesador de Intel sea malo,

ya que se pueden realizar algunas optimizaciones al código para mejorar su rendimiento, como es el caso de utilizar *schedule(guided,4)* que básicamente le indica al código que asigne bloques de trabajo a los hilos conforme terminan su ejecución. Lo que permite que los núcleos más rápidos trabajen más que los lentos, viéndose reflejado en la considerable mejoría que existe al comparar las aceleraciones de los núcleos 3. Sin embargo, estas opciones se deben de modificar para cada caso específico.

Por otro lado, si bien la aceleración del CPU de intel es peor que la del de AMD, es importante notar que las curva de tiempo de computo y aceleración presentan comportamientos más suaves, y a su vez la disminución del consumo es constante con el aumento del número de núcleos.

Finalmente es importante resaltar lo obvio, comparar un CPU de laptop contra uno de escritorio es algo sencillamente injusto, siendo lo realmente importante de este texto el ejercicio de identificar las ventajas de cada arquitectura. Siendo las ventajas Ryzen su homogeneidad y potencia de computo, mientras que el Intel destaca en su tendencia en reducir el consumo al aumentar el numero de núcleos, lo que hace a las configuraciones híbridas de mucho núcleos bastante llamativas.

REFERENCIAS

-
- [1] MuyComputer. Evolución de cpus x86. https://www.muycomputer.com/2009/04/15/actualidadnoticiasevolucion-de-cpus-x86_we9erk2xxdceucoulh59l0bs0hzo7evpxproo6lg8izg_qqw0pexpbkiphpdl57j/, April 2009. Accedido: 2026-05-27.
 - [2] Apple Inc. Mac computers with apple silicon. <https://support.apple.com/en-us/116943>, May 2026. Accedido: 2026-05-27.
 - [3] NVIDIA Corporation. Nvidia vera cpu. <https://www.nvidia.com/en-us/data-center/vera-cpu/>. Accedido: 2026-05-27.
 - [4] OpenFOAM Foundation. Openfoam for macos. <https://openfoam.org/download/macros/>, July 2024. Accedido: 2026-05-27.
 - [5] SCALE Language. Compile cuda with scale. <https://docs.scale-lang.com/stable/manual/tutorials/how-to-use/>. Accedido: 2026-05-27.
 - [6] AMD Corporation. System requirements (linux). <https://rocm.docs.amd.com/projects/install-on-linux/en/latest/reference/system-requirements.html>. ROCm documentation. Accedido: 2026-05-28.
 - [7] NVIDIA Corporation. Nvidia hpc sdk current release downloads. <https://developer.nvidia.com/hpc-sdk/downloads>. Versión 26.3, bundled with CUDA 13.1. Accedido: 2026-05-28.
 - [8] Intel Corporation. Cómo funcionan los procesadores intel® core™. <https://www.intel.la/content/www/xl/es/gaming/resources/how-hybrid-design-works.html>, July 2025. Accedido: 2026-05-27.
 - [9] NanoReview. Amd ryzen 7 5700x3d: benchmarks and specs. <https://nanoreview.net/en/cpu/amd-ryzen-7-5700x3d>, January 2024. Accedido: 2026-05-27.
 - [10] Intel Corporation. Intel® core™ i5-1335u processor (12m cache, up to 4.60 ghz) - specifications. <https://www.intel.com/content/www/us/en/products/sku/232153/intel-core-i51335u-processor-12m-cache-up-to-4-60-ghz/specifications.html>, April 2022. Accedido: 2026-05-27.